MEETUP AI

Reverse engineer anything

Donner des yeux à votre agent — et des preuves

Anne-Marie Roy

AI Meetup · 5 novembre 2026 · Code Academy 123

Une question pour commencer

Pourquoi cette calculatrice affiche-t-elle 220 quand vous tapez 200 + 10 % ?

Deux conventions de calcul du pourcentage cohabitent dans les calculatrices : « +10 % du montant » ou « +0,10 ». Le manuel ne dit pas laquelle tourne sur votre appareil.

La réponse n'est pas dans la documentation. Elle est dans le binaire.

Le problème n'est pas le code. C'est l'absence de code.

Vous voyez une fonctionnalité.
Vous voulez la même.

Pas de source
Le code n'est pas fourni — et ne le sera jamais.

Pas de doc
Aucune spécification, aucun schéma d'API.

Pas d'API
Juste un binaire qui tourne, ou une app installée.

Votre agent sait écrire du code. Il ne sait pas lire un binaire.

Qu'est-ce que REA ?

Un serveur MCP. Surtout, pas une baguette magique.

« See a feature you like. Understand how it works, down to the binary level. »

README officiel du projet

229
outils exposés à l'agent, via MCP

MIT
licence permissive, usage professionnel

Local
l'analyse tourne sur votre machine

Chaque conclusion revient avec ses preuves — et avec ses limites déclarées.

Comment ça marche

Quatre verbes, pas une boîte noire.

1 · DEMANDER

Vous posez la question

en français

2 · APPELER

L'agent appelle REA via MCP :

désassembler, inspecter, tracer

3 · ANALYSER

Ghidra, Hopper, IDA, dépackers

tournent sur votre machine

4 · RÉPONDRE

Offsets, extraits, raisonnement,

et ce qui reste incertain

Aucun binaire ne quitte votre machine. Seul le modèle de raisonnement peut être distant — appairez-le à un LLM local pour un air-gap complet.

Trois idées qui rendent l'outil crédible

Ce qui sépare un agent outillé d'un chatbot qui devine.

MCP

Un protocole, pas un prompt. Le setup enregistre REA comme serveur MCP et installe les instructions de travail qui vont avec.

Analyse locale

Désassembleurs, décompilateurs et dépackers tournent sur votre matériel. Ni le binaire, ni les résultats ne sont téléversés.

Contrat de preuve

Chaque réponse arrive avec ses preuves et la liste explicite de ce qui n'a pas pu être déterminé.

Sans le troisième, les deux premiers ne sont qu'un wrapper de plus.

Ce que REA sait ouvrir aujourd'hui

Huit familles de cibles, une seule interface.

Binaires natifs

JavaScript / Electron

APK Android

.NET & CIL

WASM & bytecode EVM

Firmware IoT

Artefacts de modèles

Captures réseau

Ce n'est pas un outil de crack. C'est un outil de lecture.

Une couche au-dessus, pas un remplacement

Les outils classiques font le gros du travail. REA ajoute le raisonnement.

Ghidra	décompilateur open source de la NSA depuis 2019	**REA le pilote pour le travail natif**
IDA Pro	standard commercial du secteur	**se branche sur une installation existante**
Hopper	désassembleur macOS	**le setup peut l'installer pour vous**
Frida	instrumentation dynamique	**REA reste plutôt statique et navigateur**

Démo 1 — la calculatrice

Source : calc.c —> compilé en calc

Même source, deux binaires

Une ligne de commande décide de ce que l'analyste verra dans Ghidra

B U I L D R E L E A S E

```
gcc -O2 -s calc.c -o calc_release
```

- Inlining, appels absorbés, variables en registres
- -s supprime la table des symboles, même posée avec -g
- Dans Ghidra : FUN_00101070, local_18, aucun repère

✕ **Opaque — plus aucun nom**

B U I L D D E B U G

```
gcc -O0 -g calc.c -o calc_debug
```

- Aucune optimisation : l'assembleur suit le C ligne par ligne
- Symboles et variables embarqués, avec les numéros de ligne
- Dans Ghidra : main, apply — dont la règle de pourcentage

✓ **Lisible — les noms de vos fonctions sont là**

Une seule différence de flags sépare un binaire réel d'un binaire pédagogique.

Démo 2 — la boîte noire

oracle · 33 Ko (Mach-O arm64) · stripped · aucune source

DÉMO BLACKBOX : RECONSTRUIRE, PUIS VÉRIFIER

Même entrée, même réponse ?



Binaire sans code source

Analyse avec REA

Retrouver la formule, les coefficients et les arrondis



Reconstruction autonome



500 nouvelles questions

Graine choisie après la reconstruction



Programme original : oracle

Réponses de référence



Réponses reconstruites



Comparer les 500 réponses

500 / 500 : PASS

Tolérance : 0,001

Exemple concret : Entrée : 100, 10, 0 → Oracle : 1,262 → Python : 1,262 → Correspond (écart 0,000)

Validé sur ces 500 cas. Pas une preuve pour toutes les entrées.

La reconstruction calcule elle-même : elle n'appelle pas le binaire.

Démo 3 - APK Cozmo
Étude de cas

Le robot que le Play Store a oublié

Une application disparue, bug tablette Android, un correctif d'une ligne

Le contexte — un robot orphelin

Cozmo, robot éducatif d'Anki (2016) : l'application a disparu du Play Store

2016 — ANKI LANCE COZMO

Robot éducatif à cubes connectés, SDK Python, Code Lab (blocs type Scratch). Un succès critique et public auprès des enfants, des familles et des écoles.

2019 — ANKI FERME SES PORTES

L'application est retirée du Play Store. Sans elle, les robots encore fonctionnels deviennent des briques connectées.

AUJOURD'HUI — DIGITAL DREAM LABS

Reprise du produit, versions 3.4 / 3.6 légitimes mais non listées sur le store. L'APK se récupère sur des miroirs, pas sur le Play Store.

Tablettes Android

Tablettes Android à équiper pour les cours de robotique de Code Academy 123 — avec une application qui ne s'installe plus officiellement.

Sans installer l'application officiellement, comment remettre Cozmo en classe ?

L'anatomie de l'app

Trois couches — et un robot qui ne dépend d'aucun serveur

1

COQUE UNITY

Toute la logique — interface, Code Lab, mécaniques de jeu — est en C# dans `Assembly-CSharp.dll` : 2,9 Mo, décompilable.

2

MOTEUR NATIF

Le pilotage du robot est confié à `libcozmoEngine.so` (17 Mo), qui parle au robot en WiFi direct. C'est exactement ce protocole que **PyCozmo** a réimplémenté en Python open source.

3

AUCUN SERVEUR

L'app demande les permissions WiFi, Bluetooth et multicast, puis ouvre sa connexion directement au robot. Rien ne transite par Anki ni par Digital Dream Labs.

Parce qu'elle ne dépend d'aucun serveur, l'app reste réparable — et le robot reste en classe.

La méthode REA — quatre passes

Récupérer → Cartographier → Instrumenter → Corriger

1 **Récupérer**

APK et OBB de Cozmo 3.6.6 depuis des miroirs légitimes (Digital Dream Labs), signature du paquet vérifiée avant toute installation.

2 **Cartographier**

Lecture du manifeste, du binaire IL2CPP et des actifs. Découverte clé : l'interface **Code Lab** est une **WebView Unity** servant du Scratch (Blockly) en file://.

3 **Instrumenter**

JavaScript patché à chaud sur la tablette via **adb** ; les événements de la WebView remontent vers logcat grâce au pont de messagerie Unity (cozmoDASLog).

4 **Corriger et valider**

Cause identifiée dans les logs, correctif d'une ligne, test sur le robot : Cozmo prononce « Bonjour ». Puis kit reproductible pour les dix tablettes.

CE QUI A MARCHÉ

— Le débogage sans fil adb sur tablette Android
— Le rechargement à chaud des fichiers JS de la WebView
— Les logs comme seule source de vérité sur le bug
— Un correctif chirurgical, réversible, documenté

LES PIÈGES ÉVITÉS

— Fausse piste mémoire : les crashs venaient de Gboard, pas du bug du clavier
— La console WebView n'atteint pas logcat : il a fallu détourner le pont Unity
— Une première capture effacée par un reset trop tôt

La démo — du log capturé au « Bonjour »

Le symptôme, la preuve, le correctif — une chaîne causale complète

1 **Symptôme — un clavier qui s'évapore**

Dans Code Lab, toucher le texte « Hi » du bloc Say ouvre le clavier, qui disparaît aussitôt. Créer une variable, elle, fonctionne.

2 **Cause racine — resize → hideChaff**

À l'ouverture du clavier, la WebView émet window.resize. Blockly y répond par hideChaff(), qui détruit tout éditeur de champ ouvert.

3 **Fix — une ligne, chirurgicale**

Dans blockly_compressed_vertical.js : si le champ texte a le focus, le resize lié au clavier ne ferme plus l'éditeur. Voir le diff ci-contre.

Deux heures d'investigation → 1 ligne de JS → Cozmo dit « Bonjour »

PREUVE — LOG CAPTURÉ SUR LA TABLETTE

```
19:44:18.699  showEditor_ called
19:44:18.822  window RESIZE
19:44:18.840  WidgetDiv.hide  ← hideChaff
19:44:18.841  input BLUR
19:44:19.328  activeElement = BODY
+600 ms       champ détruit, clavier fermé
```

LE DIFF — GESTIONNAIRE window.resize

```
- on resize : Blockly.hideChaff(!0)
-   Blockly.svgResize(a)
+ on resize : si le champ a le focus,
+   garder l'éditeur ouvert
```

La démo — REA en trois commandes

Voie CLI complète : rea setup non applicable ici, installation faite à la main

1 **Installer l'outil — une seule fois**

npm install -g rea-agents (6.8.0), puis brew install openjdk jadx pour le moteur de décompilation Android.

2 **Brancher le pont MCP headless**

C'est le seul réglage : export REA_JADX_MCP_JAR pointe vers le JAR jadx-headless-mcp 0.7.1, téléchargé et vérifié par nos soins — REA ne le télécharge pas.

3 **Inspecter l'APK sans l'exécuter**

rea inspect-android-package apk/cozmo-3.6.6.apk — manifeste, permissions, SDK min/cible. Sortie JSON « Evidence », exploitable par n'importe quel agent.

Trois commandes copiables en direct pendant le Meetup

LE TERMINAL — CE QUI A ÉTÉ TAPÉ

```
$ npm install -g rea-agents
$ brew install openjdk jadx
$ export REA_JADX_MCP_JAR=
    $PWD/tools/jadx-headless-mcp-0.7.1-all.jar
$ rea doctor # ✅ tout vert
$ rea inspect-android-package \
    apk/cozmo-3.6.6.apk
```

À RETENIR

— rea setup existe, mais cible les agents supportés (Kimi absent).

— REA vérifie l'empreinte du JAR et refuse un pont modifié.

— rea setup ne fournit jamais le pont JADX : manuel de toute façon.

Cozmo - sous-systèmes décompilés
La reconnaissance faciale

Le robot que le Play Store a oublié

Ce que le robot voit — et où ça se décide

Le C# reçoit un visage déjà identifié · Omron OKAO FaceLib dans le moteur natif

ROBOT

caméra
flux JPEG
WiFi direct

→

MOTEUR NATIF

libcozmoEngine.so
appelé par P/Invoke

→

OKAO FACELIB

détection · repères
expression

→

ALBUM

10 visages
5 photos chacun

→

CODE LAB

Robot.cs → écrans
visage déjà identifié

10 visages dans l'album

5 photos par visage

5 expressions reconnues

Firmware du robot non analysé — localisation déduite du binaire natif embarqué dans l'APK.

Tout se calcule sur la tablette. Rien ne part sur un serveur.

Démo 4 - Jeu - Dinosaure

Dinorunner

clone du Chrome T-Rex Runner (code Chromium open source)



REA 6.8.0 · ANALYZE_JAVASCRIPT_APPLICATION

Mode REA : reverse engineering de dinosaur.game

Analyse statique du clone du Chrome T-Rex Runner — des bundles minifiés jusqu'à la logique de jeu

1 Ciblage

Le jeu ne charge pas sur la page d'accueil : il vit dans une iframe (/fr/dinosaur-game). 6 bundles webpack téléchargés, 266 Ko de JavaScript.

2 Analyse statique REA

6/6 fichiers parsés · 84 187 nœuds d'AST visités · 422 modules webpack extraits · 1 356 constats · 0 échec de syntaxe.

3 Identification du moteur

809.js = moteur du jeu (constantes Chromium lisibles) · 77/229/54.js = bibliothèques tierces · dino_default.js = bootstrap webpack.

CONSTANTES LISIBLES DANS 809.JS

```
"CACTUS_SMALL"
"CACTUS_LARGE"
"PTERODACTYL"
"GRAVITY"
"INITIAL_JUMP_VELOCITY"
"MIN_JUMP_HEIGHT"
"SPEED_DROP_COEFFICIENT"
"SPEED"  ·  "YPOS"
"arcade-mode"  ·  "disabledEasterEgg"
```

LIMITES DU CONSTAT

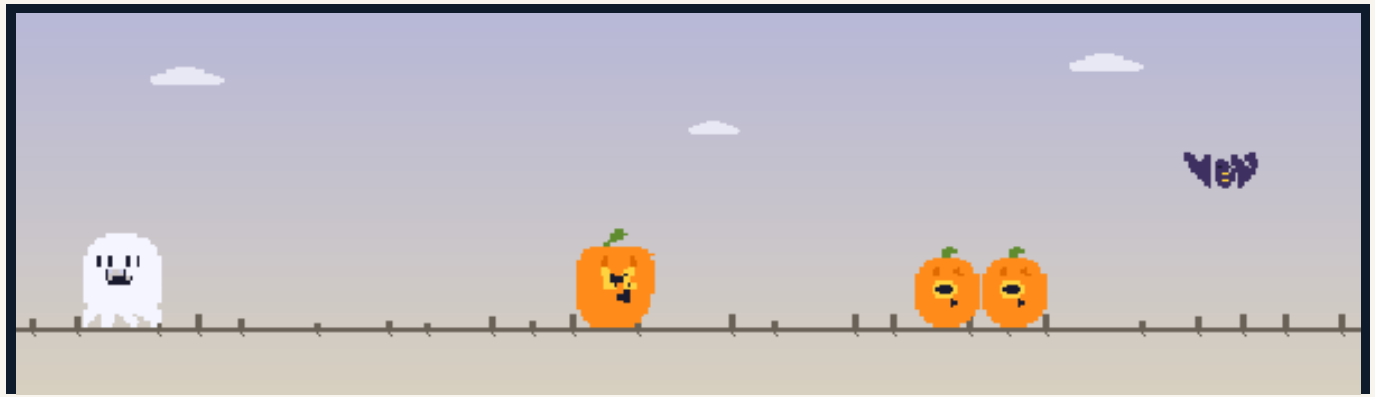
Variables minifiées (r, n, e…) · graphe sémantique partiel (plafond 20 000 nœuds/fichier) · analyse statique : le bootstrap n'a jamais été exécuté.

Cible : dinosaur.game — clone du Chrome T-Rex Runner (code Chromium open source) · Evidence complète : 89 Mo, 21 inconnus explicites

RECRÉATION · HALLOWEEN RUNNER

Même moteur, nouvelle peau

La mécanique extraite par REA, re-décorée : fantôme ou sorcière, citrouilles et chauves-souris



La scène recréée : sprites pixel art dessinés au code, rendus en canvas. Le fantôme s'élève, les citrouilles arrivent en groupe, la chauve-souris force le saut ou la baisse.

Jouable en local : index.html

L'habillage change, la mécanique ne bouge pas : c'est la preuve que la logique extraite était la bonne.

REMPLACEMENTS

FANTÔME OU SORCIÈRE — au choix, 2 frames de course + posture baissée

CITROUILLES SCULPTÉES — petites (1 à 3 groupées) et grosses, chacune sa hitbox

CHAUVES-SOURIS — 3 hauteurs (sauter / baisser / libre), dès la vitesse 8

MÉCANIQUE REPRISE DU RUNNER

gravité 0.5 · impulsion 11 · vitesse 6 → 13 (+0.001/frame) · collision par boîtes · jour/nuit tous les 700 pts

nouveau : flottement — espace maintenu = chute à 22 % de gravité

Canvas 600×170 @ 60 fps (pas de temps fixe) · WebAudio synthétisé · record en localStorage · projet : halloween-runner/

Autres Etudes de Cas Officielles

Cinq études de cas publiées par REA

Toutes vérifiables : code, figures et résultats contrôlés.

Une analyse avec les preuves

Aegis	APK Android	« Comment cette application fabrique-t-elle le code à six chiffres qui change toutes les 30 secondes ? » L'analyse retrouve le calcul à partir du temps et de la clé du compte, puis le reproduit dans une démonstration. Étude Aegis
CTF DownUnderCTF	binaire natif	« Comment un binaire dépouillé vérifie-t-il le drapeau sans jamais le révéler ? » L'analyse reconstruit les 16 tours de transformations appliqués à la saisie dans un ELF x86-64 stripé, inverse la fonction de brassage et fait apparaître le drapeau dans une démonstration. Étude DUCTF
DX-Ball (1996)	jeu Windows	« Quand un événement se produit à gauche ou à droite de l'écran, comment le jeu répartit-il le son entre les haut-parleurs ? » Une petite fonction est reconstruite en C. Sa compilation reproduit les **63 octets de la fonction étudiée**, pas ceux du jeu entier. Étude DX-Ball
Notion	application Electron	« Que se passe-t-il quand je copie du contenu ? » L'analyse suit le trajet entre l'interface et le processus principal. **IPC** désigne les messages échangés entre ces processus. Étude Notion
Touhou TH04	DOS PC-98, 16 bits	« Comment le jeu calcule-t-il la direction d'un anneau de projectiles vers le joueur ? » L'analyse retrouve les calculs d'angles dans les instructions 16 bits et les confronte à une reconstruction. Étude TH04

Du jeu de 1996 à l'app Electron : le goulot n'est plus « puis-je décoder »=. Le défi est aussi de savoir sur quelles preuves repose la reconstruction.

ATTENTION

Un binaire non fiable est une entrée non fiable.

Les chaînes de caractères et les symboles du fichier analysé arrivent dans le contexte de l'agent. C'est une surface d'injection de prompt : un binaire peut contenir des instructions qui visent votre modèle.

Analysez en bac à sable. Lisez les preuves avant d'agir sur la réponse.

Le cadre, avant les démos

La slide que personne ne met dans son talk.

Généralement admis

— Défis CTF et recherche (CTF : Capture The Flag)

— Logiciels libres et open source

— Votre propre logiciel

— Interopérabilité avec ce que vous possédez légalement

Risqué ou illégal

— Contourner un DRM (Digital Rights Management). En pratique, le DRM agit comme un verrou numérique qui exige souvent une clé de déchiffrement ou une connexion à un serveur pour autoriser l'accès ou l'utilisation du contenu acheté ou loué.

— Retirer un contrôle de licence

— Cloner le produit d'un concurrent

— Redistribuer du code décompilé

Directive 2009/24/CE, art. 5 et 6 · Code de la propriété intellectuelle, L122-6-1.

Ceci n'est pas un avis juridique.

Où ça échoue

Un outil qu'on surestime coûte plus cher qu'un outil qu'on n'a pas.

- Projet jeune : l'API et les contrats d'outils bougent encore.
- L'analyse native est lente sur un binaire de 200 Mo — des minutes, pas des secondes.
- Le pseudocode reste parfois incomplet : il faut un moteur externe (Ghidra, Hopper, IDA).
- L'obfuscation lourde (packers, VM-based) n'est pas résolue par magie.
- Aucune garantie d'exhaustivité : l'agent dit ce qu'il a trouvé, pas tout ce qui existe.

Ce que ça change, concrètement

Quatre situations où l'outil paie sa place.

Legacy

documenter un système dont le développeur est parti il y a huit ans.

Sécurité

vérifier ce qu'un binaire fait vraiment, pas ce qu'il prétend faire.

Produit

comprendre une fonctionnalité pour en écrire une version propre.

Pédagogie

montrer ce qu'est réellement un programme compilé.

Prêt en quelques minutes

Rien à compiler, rien à héberger.

```
$ npx rea-agents setup
```

installe REA et l'enregistre pour votre agent

```
$ rea doctor --client codex --json
```

vérifie que les outils MCP sont bien là

```
$ npx -y rea-agents@latest analyze-javascript-application ./app --json
```

sans agent, sortie JSON

· Un désassembleur pour le natif (Ghidra est gratuit).

· Un agent : Claude Code, Codex, Cursor, Gemini CLI.

· Un fichier — ou une session de navigateur.

Ressources — la carte de route

Six portes d'entrée, du premier export à la lecture des papiers.

Commencer

Le dépôt REA (code, doc, releases), le site officiel, le guide d'installation pour votre agent, et un premier export : bouton → IPC → CSV.

Regarder

« Use REA to clone any software » · le tutoriel T-Rex, pas à pas · Ghidra pour débutants · Frida pour Android.

Livres

The Ghidra Book, 2e éd. · Reverse Engineering for Beginners (gratuit) · Practical Binary Analysis · Practical Malware Analysis.

Papiers

LLM4Decompile · AutoDecompile · SALT4Decompile · DIRTY (noms de variables) · T-Rex · agents face à la prompt injection dans le binaire.

Pratiquer

OpenSecurityTraining2 x86-64 · pwn.college (shell, mémoire, Python) · Malware Unicorn RE101 · Nightmare · crackmes.one.

Fouiller

Awesome-REA, 3 309 entrées · Dembrandt · mitmproxy2swagger · JADX-AI-MCP · BSim pour la signature de fonctions.

L'ordre qui marche : installer → un export → une app Electron → un binaire → une fonction réelle → Ghidra seulement à la fin.

Demandez en une phrase.
Vérifiez en ingénieur.

Quand la compréhension d'un logiciel devient banale, le jugement devient la compétence rare.

Ensuite : démo en direct · questions

Merci.

Anne-Marie Roy · Code Academy 123

github.com/royam0820/Jev-Meetup

rea.tools · github.com/morluto/rea

github.com/reacher-z/Awesome-REA